

ESPECIFICACAO DO PROTOCOLO TCP CPF/CNPJ

Versao 1 - 20/09/2026

1. Transporte e enquadramento

O protocolo usa TCP sobre IPv4 ou IPv6, codificacao UTF-8 e mensagens JSON. Cada mensagem ocupa uma linha e termina com LF (0x0A). O servidor deve processar uma linha por vez e nunca concatenar respostas na mesma linha. Porta padrao: 5000. O cliente inicia a conexao TCP.

2. Regras gerais

- Todo objeto possui type. Mensagens de aplicacao usam version=1.
- request_id e uma string opaca escolhida pelo emissor para correlacionar requisicao e resposta. O servidor devolve o mesmo request_id.
- Campos desconhecidos devem ser ignorados; campos obrigatorios ausentes causam uma resposta error.
- Valores podem conter pontuacao; a validacao remove todos os caracteres nao numericos antes de aplicar o algoritmo.

3. Abertura da sessao

Cliente -> Servidor: {"type":"hello","version":1,"request_id":"id"}

Servidor -> Cliente: {"type":"hello_ack","version":1,
"request_id":"id","service":"cpf-cnpj-validator"}

A primeira mensagem deve ser hello. Versao diferente de 1 encerra a sessao apos error com code=UNSUPPORTED_VERSION.

4. Mensagens de validacao

Cliente -> Servidor: {"type":"validate","version":1,
"request_id":"id","document_type":"CPF","value":"529.982.247-25"}
document_type e obrigatoriamente CPF ou CNPJ; value e string.

Servidor -> Cliente: {"type":"validation_result","version":1,
"request_id":"id","document_type":"CPF","value":"52998224725",
"valid":true,"message":"CPF valido"}
value na resposta e o valor normalizado com somente digitos.
valid=false significa documento sintaticamente processado, mas invalido.

5. Encerramento e erros

Cliente -> Servidor: {"type":"quit","version":1,"request_id":"id"}
Servidor -> Cliente: {"type":"bye","version":1,"request_id":"id"}

Erro: {"type":"error","version":1,"request_id":"id-ou-null",
"code":"CODIGO","message":"descricao legivel"}
Codigos: INVALID_JSON, INVALID_MESSAGE, INVALID_HELLO, HELLO_REQUIRED,
UNSUPPORTED_VERSION, INVALID_REQUEST_ID, INVALID_VALIDATE,
MESSAGE_TOO_LARGE e SERVER_BUSY.

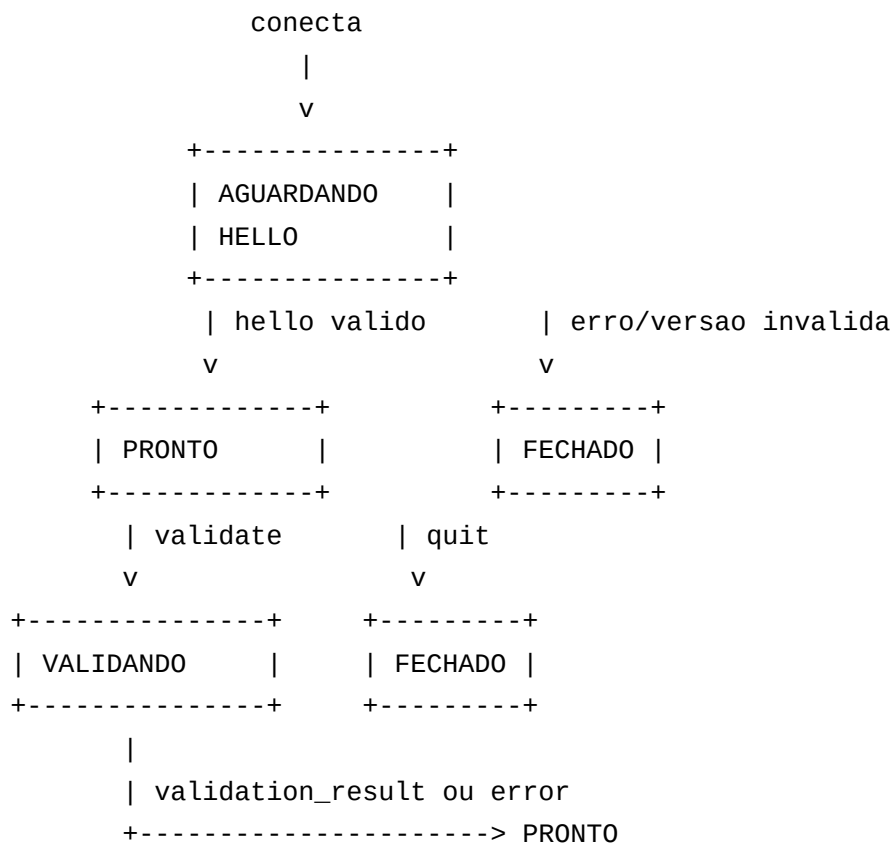
6. Concorrencia e protecao contra DoS

O servidor cria uma thread por conexao aceita. O conjunto dinamico
active_sockets associa um identificador a cada socket ativo e e protegido
por lock. Um semaforo limitado controla max_connections (padrao 32).
Quando o limite e atingido, o servidor envia SERVER_BUSY, fecha o socket
e nao cria thread. Cada conexao tambem tem timeout de leitura de 30 segundos.

7. Validacao

CPF: 11 digitos, rejeicao de todos os digitos iguais e verificacao dos dois
digitos de controle pelo modulo 11. CNPJ: 14 digitos, mesma rejeicao e
pesos oficiais (5,4,3,2,9,8,7,6,5,4,3,2 e 6,5,4,3,2,9,8,7,6,5,4,3,2).

8. Diagrama de estados finitos do protocolo



Transicoes adicionais:

- AGUARDANDO HELLO --HELLO_REQUIRED/INVALID_HELLO--> FECHADO.
- PRONTO --mensagem desconhecida--> PRONTO (responde UNKNOWN_MESSAGE_TYPE).
- Qualquer estado --EOF, timeout ou falha de socket--> FECHADO.
- Listener sem vaga --nova conexao--> SERVER_BUSY e FECHADO.

9. Exemplo de sessao completa

```

C> {"type":"hello","version":1,"request_id":"a"}
S> {"type":"hello_ack","version":1,"request_id":"a",
    "service":"cpf-cnpj-validator"}
C> {"type":"validate","version":1,"request_id":"b",
    "document_type":"CNPJ","value":"11.222.333/0001-81"}
S> {"type":"validation_result","version":1,"request_id":"b",
    "document_type":"CNPJ","value":"11222333000181",
    "valid":true,"message":"CNPJ valido"}
C> {"type":"quit","version":1,"request_id":"c"}
S> {"type":"bye","version":1,"request_id":"c"}

```